

**Group Project: NU Industries Production Planning Optimization
Final Report**

Samin Bahizad
Bobby Hendricks
Deysi Paniagua-Perez
Nathan Stark
MSDS 460: Decision Analytics
August 30, 2026

Table of Contents

Abstract.....	3
Introduction.....	4
Literature Review.....	4
Methodology.....	5
Model Objective.....	5
Decision Variables.....	5
Model Constraints.....	6
Model Assumptions.....	6
Model Verification and Validation.....	6
Computational Experiment and Results.....	7
Baseline Results.....	7
Scenario Analysis.....	8
Raw Material 1 Sensitivity Analysis.....	9
Integer Programming Results.....	11
Discussion and Conclusions.....	11
References.....	13
Appendix A. Detailed Model Results.....	14
Appendix B. Verification and Validation.....	17
Appendix C. Supplemental Figures.....	18
Appendix D. Python Programming Code.....	19

Abstract

NU Industries has to coordinate production, inventory, labor, raw materials, advertising, and shipment across two plants and five production periods. This project report presents a deterministic linear programming model created to identify the operating plan that maximizes total profit while satisfying contract demand and the company's operating constraints. The model was developed in Python using PuLP and solved with GLPK. A baseline model was solved, after which scenario and sensitivity analyses were used to examine the effects of additional advertising, raw materials, and regular labor capacity. An integer programming model was also used to evaluate the impact of requiring whole-unit production. The baseline model yielded a maximum profit of approximately \$7.07 million. Raw Material 1 was the most crucial resource bottleneck with a 5 percent increase capturing almost all of the available profit growth. Additional regular labor at Plant B offered a smaller benefit, while expanding the advertising budget had no effect. Requiring whole-unit production reduced profit by only approximately \$254. Overall, results demonstrate how NU Industries can use optimization to balance production-related decisions and prioritize the constraints with the greatest financial impact.

Keywords: Linear programming, production planning, sensitivity analysis, inventory optimization, integer programming

Introduction

Production planning involves organizations managing multiple decisions that influence each other. For instance, higher unit production in one period may entail added labor and raw materials, while earlier production may increase inventory holding costs but help avoid more expensive production in the future. Similarly, advertising may create extra demand and revenue; however, the organization must have sufficient production capacity and resources to fulfill that demand. Balancing these tradeoffs makes production planning well suited for optimization.

NU Industries manufactures three products, Widgets, Gadgets, and Flugels, at two plants over five production periods. The company must fulfill its existing contract demand while evaluating if advertising should generate additional demand. Production requires two raw materials and labor, with each plant having different resource requirements, inventory capacities, and labor cost increases. Products are then shipped from both plants to a distribution center.

This study aims to identify the production, inventory, advertising, labor, and shipping plan to maximize NU Industries' total profit while fulfilling demand and operating constraints. The study is broken down into several phases. First, it seeks to determine the best production and operating decision combination that generates the highest profit during baseline conditions. Next, it identifies the most restrictive resources that limit additional profitable production. It also strives to determine whether profit is significantly improved when increasing advertising budget, raw material availability, or labor capacity. Finally, it evaluates whether the optimal solution is affected by requiring production quantities to be whole units.

Literature Review

Aggregate production planning (APP) is often used to coordinate production, inventory, workforce, and capacity decisions across a defined planning horizon. Research stresses that mathematical programming can support production planning by assessing several decisions at the same time. These decisions include when to produce, how much labor and resources are accessible, how much inventory to retain, and how demand will be met (Nam and Logendran 1992). Linear programming has also been used in manufacturing to plan production in multiple periods while taking labor, overtime, production, and inventory costs into account (Tang, Adulbhan, and Zubair 1981). One advantage of these approaches is that they can evaluate several connected planning decisions within the same model, which makes them useful for structured production environments with known inputs. However, their results depend on the accuracy of the demand, cost, and capacity values used in the model. This is similar to the NU Industries problem because production, inventory, labor, and resource decisions are connected across the different periods.

Capacity and inventory decisions are also essential components of production planning. Earlier inventory production may generate additional holding costs, but it can also diminish the effect of higher production costs or limited capacity. Therefore, more advanced APP models integrate other operational constraints such as capacity changes and multiple production locations (Zhang et al. 2012; Mirzapour Al-e-hashem et al. 2011). These approaches can provide a more realistic representation of multi-site manufacturing plants, yet also require more complex model formulation along with additional required parameters. This added detail can improve the realism of the planning model, but it also increases the amount of data and computational effort needed to develop and solve it.

Another important limitation of traditional deterministic APP models is uncertainty in demand. Possibilistic linear programming and fuzzy models can account for uncertain manufacturing conditions, although these methods require more assumptions, data, and modeling complexity (Wang and Liang 2005; Mirzapour Al-e-hashem et al. 2011). Other studies have also used fuzzy linear programming to account for changes in customer demand and production output (Djordjevic, Petrovic, and Stojic 2019). These models can represent production systems with multiple sites, capacity decisions, and a larger number of operational relationships, still they also require additional parameters and more complex formulations. The NU Industries model uses a more deterministic linear programming approach since the case study provides fixed demand, costs, capacities, and resource requirements. The analysis is expanded through sensitivity analysis and integer programming to evaluate how the recommended solution is affected by key factors. Therefore, the study follows the same multi-period production-planning framework found in prior APP research, while focusing on a deterministic setting and using scenario, sensitivity, and integer programming analyses to test the stability and practicality of the solution.

Methodology

Model Objective

The NU Industries production planning problem was modeled as a linear programming model in Python using PuLP. The objective is to maximize total profit while meeting demand and staying within the company's operating limits over five production periods. Total profit is calculated as sales revenue minus raw material, regular labor, overtime labor, inventory holding, transportation, and advertising costs. The model includes three products, Widgets, Gadgets, and Flugels, and two manufacturing plants, Plants A and B. Contract demand must be satisfied in each period, while advertising can be purchased during Periods 1 through 4 and can increase demand in the following period.

Decision Variables

The baseline model includes decision variables for units produced by product, plant, and period; units shipped from each plant to the distribution center; ending inventory; regular and

overtime labor hours; and additional demand generated through advertising. All baseline decision variables are continuous and nonnegative. Production is then limited to whole units in a separate integer programming model.

Model Constraints

Inventory balance constraints require the beginning inventory plus current production to equal shipments plus ending inventory. Demand constraints require total shipments to equal contract demand plus any extra demand created by advertising in the previous period.

Labor constraints call for coverage of production labor by regular or overtime hours. Moreover, Plant A has 2,500 regular labor hours available per period, while Plant B has 3,800 hours. Overtime is allowed when additional labor is needed.

Raw Material 1 is restricted to 140,000 pounds per period, whereas Raw Material 2 is capped at 5,000 pounds per period across both plants. Inventory is limited to 70 total units at Plant A and 50 total units at Plant B. The advertising budget is limited to \$70,000 across the planning horizon. Finally, the model assumes no beginning inventory and requires an ending inventory of zero after Period 5.

Model Assumptions

Several assumptions helped simplify the production planning problem. For example, demand, selling prices, resource requirements, and costs are treated as known values. All demand must be met, meaning shortages and backorders are not allowed. Advertising purchased in one period increases demand only in the following period. Raw materials are bought and used in the same period since the model does not incorporate raw material inventory. Overtime labor is unlimited, and fixed plant overhead is constant, because it remains the same based on the production plan. The baseline linear programming model also allows fractional production quantities.

Model Verification and Validation

The baseline solution was reviewed to ensure that the main constraints and model calculations were working properly. Checks for advertising spending, raw material limits, regular labor capacity, ending inventory, storage capacity, demand satisfaction, and inventory balance were included throughout validation. A summary of the validation results is provided in Table B.1 of Appendix B.

Other checks were performed to analyze how inventory affects the overall production plan. This analysis looked at the cost of producing Gadgets before and after the price increases to see if carrying inventory into later periods could reduce total costs. Producing Gadgets at Plant B in Period 2 and holding them for one period would cost roughly \$2.88 less per unit than producing them in Period 3. Early production at Plant A was slightly pricier when only labor and

holding costs were accounted for, suggesting that other capacity limits may impact the model's early production decisions.

The Period 3 bottleneck analysis demonstrated that both plants use all available regular labor during that period. Plant B also uses approximately 266.9 overtime hours. These results help illustrate why the model produces some Gadgets earlier and carries inventory forward to reduce production pressure in Period 3. Detailed regular and overtime labor usage by plant and period is presented in Table A.3 and Figure A.1 of Appendix A.

Computational Experiment and Results

The computational experiments began with the baseline linear programming model and then changed key parameters to measure their effect on the optimal solution. The baseline model was solved first, followed by separate tests of the advertising budget, Raw Material 1, Raw Material 2, and regular labor capacity at each plant. A combined scenario increased both Raw Material 1 and Plant B labor, and a separate sensitivity analysis tested smaller and larger increases in Raw Material 1. Finally, an integer programming model was used to determine whether requiring whole-unit production materially changed the recommended solution. The baseline model returned an optimal solution, and all validation checks for advertising, raw materials, labor, ending inventory, storage capacity, demand satisfaction, and inventory balance passed. The Python code used to formulate, solve, and validate the model is provided in Appendix D.

Baseline Results

The baseline model produced an optimal total profit of \$7,066,036.78. Sales revenue totaled \$8,480,608.45. The largest operating cost was raw material at \$965,351.24, followed by regular labor at \$334,324.10. Overtime labor cost \$29,221.61, transportation cost \$18,557.68, inventory holding cost \$1,016.69, and advertising cost \$66,100.34. Table 1 summarizes the baseline financial results.

Table 1. Baseline Financial Results

Component	Amount
Sales Revenue	\$8,480,608.45
Raw Material Cost	(\$965,351.24)
Regular Labor Cost	(\$334,324.10)
Overtime Labor Cost	(\$29,221.61)
Inventory Holding Cost	(\$1,016.69)
Transportation Cost	(\$18,557.68)

Advertising Cost	(\$66,100.34)
Maximum Profit	\$7,066,036.78

The production plan also showed several operating patterns. Flugels were produced entirely at Plant B, while Gadgets were concentrated at Plant A, with 50 units produced at Plant B in Period 1. The model carried Gadget inventory forward to manage later capacity requirements. At the end of Period 1, Plant A held 70 Gadgets and Plant B held 50; Plant A then carried 63 Gadgets at the end of Period 2 before the inventory was used in Period 3. Widgets were produced at both plants without carrying Widget inventory between periods. The complete production, shipment, and ending inventory plan is provided in Table A.1 of Appendix A.

Resource use confirms that the baseline solution is constrained by both materials and labor. Raw Material 1 was essentially at its 140,000-pound capacity in Periods 2 through 5, while Raw Material 2 reached its 5,000-pound capacity in Period 2. Plant A did not require overtime, but Plant B used approximately 1,116.2 overtime hours in Period 2, 266.9 in Period 3, 159.1 in Period 4, and 169.2 in Period 5. Advertising spending was approximately \$66,100.3 of the \$70,000 budget, leaving about \$3,899.7 unused. Advertising was purchased only for Flugels in Periods 1 through 4, creating approximately 367.2 units of additional demand in the subsequent periods (see Table A.2 in Appendix A). Therefore, the unused advertising budget suggested that another operating constraint was limiting the value of additional advertising.

Scenario Analysis

The scenario experiments tested whether additional advertising, material, or labor capacity would improve profit. Increasing the advertising budget from \$70,000 to \$100,000 produced no change in either advertising spending or maximum profit. In contrast, increasing Raw Material 1 by 25 percent raised profit by \$63,295.4, while the same increase in Raw Material 2 raised profit by only \$1,057.8. Increasing regular labor capacity by 25 percent at Plant A increased profit by \$1,697.7, compared with \$20,558.9 at Plant B. These results identify Raw Material 1 as the most important individual bottleneck and Plant B labor as the more valuable labor-capacity increase.

Table 2. Scenario Comparison

Scenario	Maximum Profit	Change from Baseline
Baseline	\$7,066,036.8	\$0.0
Advertising Budget = \$100,000	\$7,066,036.8	\$0.0
Raw Material 1 +25%	\$7,129,332.2	+\$63,295.4
Raw Material 2 +25%	\$7,067,094.6	+\$1,057.8
Plant A Regular Labor +25%	\$7,067,734.5	+\$1,697.7

Plant B Regular Labor +25%	\$7,086,595.7	+\$20,558.9
RM1 +25% and Plant B Labor +25%	\$7,131,971.6	+\$65,934.8

The combined scenario increased both Raw Material 1 and Plant B regular labor capacity by 25 percent. Maximum profit increased to \$7,131,971.6, or \$65,934.8 above the baseline. This was only \$2,639.4 more than the Raw Material 1 increase by itself. As a result, most of the combined improvement came from relieving the Raw Material 1 constraint rather than from the additional Plant B labor.

Raw Material 1 Sensitivity Analysis

Because Raw Material 1 produced the largest improvement, additional sensitivity analysis tested increases from 5 percent through 50 percent.

Table 3. Raw Material 1 Sensitivity Results

RM1 Change	Capacity (lbs./period)	Maximum Profit	Profit Increase
Baseline	140,000	\$7,066,036.8	\$0.0
+5%	147,000	\$7,129,318.5	+\$63,281.8
+10%	154,000	\$7,129,330.2	+\$63,293.4
+15%	161,000	\$7,129,332.2	+\$63,295.4
+20%	168,000	\$7,129,332.2	+\$63,295.4
+25%	175,000	\$7,129,332.2	+\$63,295.4
+50%	210,000	\$7,129,332.2	+\$63,295.4

The results show that the profit improvement occurs almost entirely with a small increase in capacity. A 5 percent increase, from 140,000 to 147,000 pounds per period, raised maximum profit to \$7,129,318.5, an improvement of \$63,281.8. Increasing capacity to 154,000 pounds added only another \$11.7 of profit, and the maximum gain of approximately \$63,295.4 was reached by the 15 percent level. No additional improvement occurred at the 20, 25, or 50 percent levels.

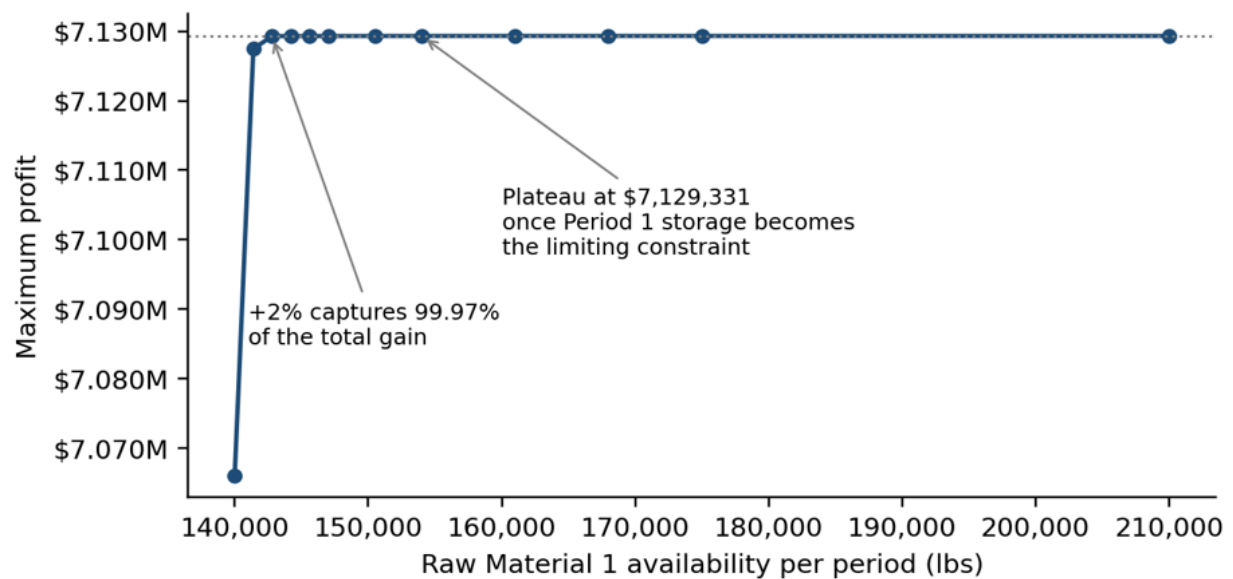


Figure 1. Maximum Profit by Raw Material 1 Capacity

The follow-up test at the 5 percent Raw Material 1 increase further explains the leveling effect. Period 2 continued to use essentially all 147,000 pounds of available Raw Material 1, while the remaining periods had unused Raw Material 1 capacity. Raw Material 2 also remained relatively tight in later periods, with only about 32.2 pounds of unused capacity in Period 5. Detailed resource usage for the 5 percent Raw Material 1 increase is presented in Table B.2 of Appendix B. Therefore, once the initial Raw Material 1 shortage was relieved, other resource and product-mix constraints limited the value of additional Raw Material 1.

Integer Programming Results

The integer programming experiment restricted production quantities to whole units while keeping shipment, inventory, labor, overtime, and advertising variables continuous. The optimal integer solution produced a maximum profit of approximately \$7,065,782.9, which was only \$253.90, or about 0.004 percent, below the baseline linear programming solution. A separate validation check confirmed that all production quantities in the integer solution were whole units. The small difference indicates that the fractional production assumption in the baseline model has almost no effect on the overall profitability or usefulness of the recommended production plan.

Table 4. Baseline LP and Integer Programming Comparison

Scenario	Maximum Profit	Change from Baseline
Baseline	\$7,066,036.8	\$0.0
Integer Programming	\$7,065,782.9	-\$253.9

Discussion and Conclusions

The baseline model resulted in a maximum profit of approximately \$7.07 million while meeting all production and demand requirements. The results also show how production and inventory decisions are connected across the five periods. For example, Gadget inventory is carried into Period 3, when regular labor capacity is fully used at both plants. Producing some Gadgets earlier helps reduce the amount that must be produced during this period.

The scenario analysis identified Raw Material 1 as the resource with the greatest impact on profit. Increasing its capacity by 25 percent increased profit by about \$63,295. This was much larger than the increases from additional Raw Material 2 or labor capacity. The sensitivity analysis also showed that a 5 percent increase in Raw Material 1 captured almost all of this increase in profit. Specifically, the 5 percent increase generated approximately \$63,282 of the maximum \$63,295 improvement observed in the sensitivity analysis. Larger increases resulted in very little additional improvement because other constraints began to limit the solution. Additional regular labor at Plant B increased profit as well, but by a smaller amount. Increasing the advertising budget from \$70,000 to \$100,000 produced no additional profit, and the baseline solution already left approximately \$3,900 of the existing budget unused. This indicates that additional advertising budget would not improve the solution unless other production or resource constraints are relieved.

The integer programming results were very similar to the baseline results. Requiring whole-unit production reduced maximum profit by only \$254 (0.004 percent). This indicates that allowing fractional production in the baseline model had very little effect on the final results. Whole-unit production provides a more realistic production plan without meaningfully changing the expected profit.

There are a few limitations to the model that are worth noting. The model assumes that demand, prices, costs, and available resources are known for all five periods. It also assumes that overtime is always available and that shortages and backorders are not allowed. Raw materials also cannot be stored between periods. These assumptions simplify the model, but in an actual production setting some of these values would likely change over time. Future analysis could look at changes in demand or costs, limits on available overtime, or other production scenarios. It would also be useful to consider how much it would actually cost to increase Raw Material 1 capacity and whether the additional profit would justify that cost.

Overall, the results show that Raw Material 1 is the main area where NU Industries could benefit from additional capacity. A relatively small increase produced most of the available improvement in profit, while larger increases had very little additional effect. Based on the modeled profit improvements, NU Industries should evaluate a small increase in Raw Material 1 availability before considering broader labor or advertising capacity increases. However, the final decision should depend on whether the cost of obtaining additional Raw Material 1 is less than the additional profit it generates. The model also shows how production and inventory can be planned across the five periods while identifying which resources have the greatest effect on profit.

References

- Djordjevic, Ivan, Dobrila Petrovic, and Gordan Stojic. 2019. "A Fuzzy Linear Programming Model for Aggregated Production Planning (APP) in the Automotive Industry." *Computers in Industry* 110: 48–63. <https://doi.org/10.1016/j.compind.2019.05.004>.
- Mirzapour Al-e-hashem, S. M. J., H. Malekly, and M. B. Aryanezhad. 2011. "A Multi-Objective Robust Optimization Model for Multi-Product Multi-Site Aggregate Production Planning in a Supply Chain under Uncertainty." *International Journal of Production Economics* 134 (1): 28–42. <https://doi.org/10.1016/j.ijpe.2011.01.027>.
- Nam, Sang-jin, and Rasaratnam Logendran. 1992. "Aggregate Production Planning—A Survey of Models and Methodologies." *European Journal of Operational Research* 61 (3): 255–72. [https://doi.org/10.1016/0377-2217\(92\)90356-E](https://doi.org/10.1016/0377-2217(92)90356-E).
- Tang, John C. S., Pakorn Adulbhan, and Tahir Zubair. 1981. "An Aggregate Production Planning for a Heavy Manufacturing Industry." *European Journal of Operational Research* 7 (1): 22–29. [https://doi.org/10.1016/0377-2217\(81\)90046-1](https://doi.org/10.1016/0377-2217(81)90046-1).
- Wang, Reay-Chen, and Tien-Fu Liang. 2005. "Applying Possibilistic Linear Programming to Aggregate Production Planning." *International Journal of Production Economics* 98 (3): 328–41. <https://doi.org/10.1016/j.ijpe.2004.09.011>.
- Zhang, Renqian, Lankang Zhang, YiYong Xiao, and Ikou Kaku. 2012. "The Activity-Based Aggregate Production Planning with Capacity Expansion in Manufacturing Systems." *Computers & Industrial Engineering* 62 (2): 491–503. <https://doi.org/10.1016/j.cie.2011.10.016>.

Appendix A. Detailed Model Results

Table A.1: Baseline Production, Shipment, and Inventory Plan

Product	Plant	Period	Production	Shipment	Ending Inventory
Widget	Plant_A	1	61.4	61.4	0.0
Widget	Plant_A	2	0.0	0.0	0.0
Widget	Plant_A	3	89.8	89.8	0.0
Widget	Plant_A	4	80.1	80.1	0.0
Widget	Plant_A	5	83.8	83.8	0.0
Widget	Plant_B	1	8.6	8.6	0.0
Widget	Plant_B	2	125.0	125.0	0.0
Widget	Plant_B	3	95.2	95.2	0.0
Widget	Plant_B	4	109.9	109.9	0.0
Widget	Plant_B	5	116.2	116.2	0.0
Gadget	Plant_A	1	270.0	200.0	70.0
Gadget	Plant_A	2	243.0	250.0	63.0
Gadget	Plant_A	3	232.0	295.0	0.0
Gadget	Plant_A	4	245.0	245.0	0.0
Gadget	Plant_A	5	240.0	240.0	0.0
Gadget	Plant_B	1	50.0	0.0	50.0
Gadget	Plant_B	2	0.0	50.0	0.0
Gadget	Plant_B	3	0.0	0.0	0.0
Gadget	Plant_B	4	0.0	0.0	0.0
Gadget	Plant_B	5	0.0	0.0	0.0
Flugel	Plant_A	1	0.0	0.0	0.0

Flugel	Plant_A	2	0.0	0.0	0.0
Flugel	Plant_A	3	0.0	0.0	0.0
Flugel	Plant_A	4	0.0	0.0	0.0
Flugel	Plant_A	5	0.0	0.0	0.0
Flugel	Plant_B	1	140.0	140.0	0.0
Flugel	Plant_B	2	356.5	356.5	0.0
Flugel	Plant_B	3	301.9	301.9	0.0
Flugel	Plant_B	4	279.1	279.1	0.0
Flugel	Plant_B	5	274.7	274.7	0.0

Advertising Results

Table A.2: Advertising-Generated Demand and Cost

Product	Period	Additional Demand	Advertising Cost
Widget	1	0.0000	0.000
Widget	2	0.0000	0.000
Widget	3	0.0000	0.000
Widget	4	0.0000	0.000
Gadget	1	0.0000	0.000
Gadget	2	0.0000	0.000
Gadget	3	0.0000	0.000
Gadget	4	0.0000	0.000
Flugel	1	181.4810	32666.580
Flugel	2	96.9375	17448.750
Flugel	3	44.1158	7940.844
Flugel	4	44.6898	8044.164

Labor Usage

Table A.3: Regular and Overtime Labor Usage

Plant	Period	Regular Labor	Overtime Labor	Total Labor
Plant_A	1	2500.0	0.0	2500.0
Plant_A	2	1725.6	0.0	1725.6
Plant_A	3	2500.0	0.0	2500.0
Plant_A	4	2500.0	0.0	2500.0
Plant_A	5	2500.0	0.0	2500.0
Plant_B	1	1952.6	0.0	1952.6
Plant_B	2	3800.0	1116.2	4916.2
Plant_B	3	3800.0	266.9	4066.9
Plant_B	4	3800.0	159.1	3959.1
Plant_B	5	3800.0	169.2	3969.2

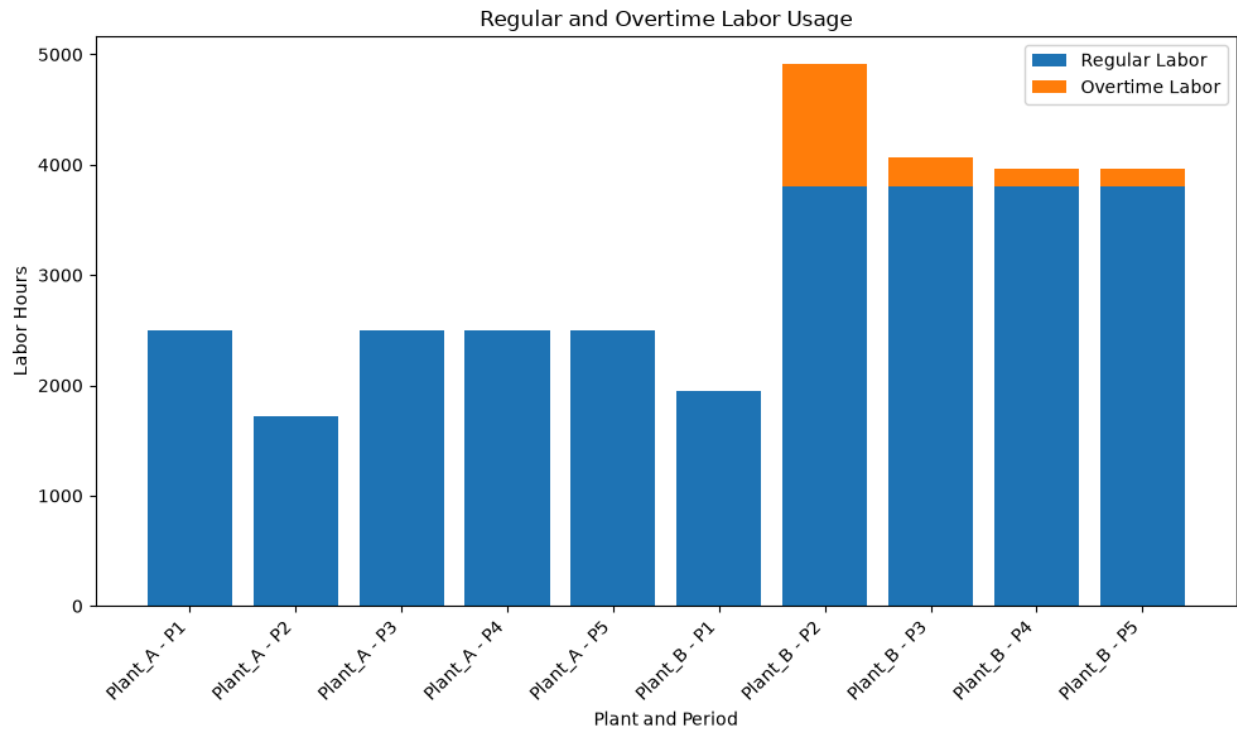


Figure A.1: Regular and Overtime Labor Usage by Plant and Period

Appendix B. Verification and Validation

Validation Summary

Table B.1: Model Validation Summary

Validation Check	Result
Advertising Budget	PASS
Raw Material Capacity	PASS
Regular Labor Capacity	PASS
Ending Inventory	PASS
Inventory Capacity	PASS
Demand Satisfaction	PASS
Inventory Balance	PASS

Raw Material 1 Follow-up Results

Table B.2: Resource Usage after a 5 Percent RM1 Increase

Period	RM1 Used	RM1 Capacity	RM1 Slack	RM2 Used	RM2 Capacity	RM2 Slack
1	105302.1	147000	41697.9	3271.8	5000	1728.2
2	147000.1	147000	-0.1	4627.8	5000	372.2
3	141674.9	147000	5325.1	4230.9	5000	769.1
4	137447.6	147000	9552.4	4552.3	5000	447.7
5	144010.8	147000	2989.2	4967.8	5000	32.2

Appendix C. Supplemental Figures

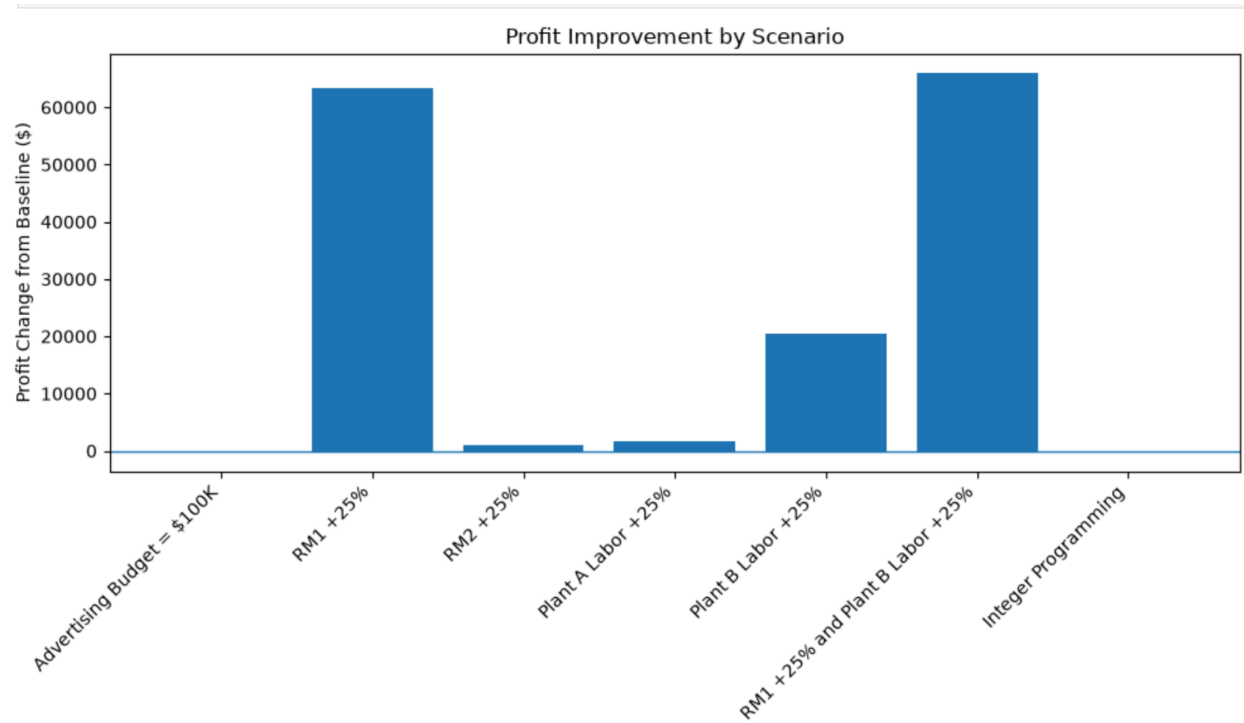


Figure C.1: Profit Change Relative to Baseline by Scenario

Appendix D. Python Programming Code

The following Python code was used to formulate and solve the baseline linear programming model, conduct verification and validation checks, evaluate alternative resource scenarios, perform the Raw Material 1 sensitivity analysis, and develop the integer programming model. Model results and supplemental output tables are presented in Appendices A through C.

D.1: Packages, Sets, and Model Parameters

```
# import pulp and pandas
from pulp import LpVariable, LpProblem, LpMaximize, LpStatus, value,
lpSum, GLPK
import pandas as pd
import matplotlib.pyplot as plt

# define variable
products = ["Widget", "Gadget", "Flugel"]
plants = ["Plant_A", "Plant_B"]
periods = [1, 2, 3, 4, 5]
advertising_periods = [1, 2, 3, 4]

# required contract demand by product and period
base_demand = {
    ("Widget", 1): 70,
    ("Widget", 2): 125,
    ("Widget", 3): 185,
    ("Widget", 4): 190,
    ("Widget", 5): 200,

    ("Gadget", 1): 200,
    ("Gadget", 2): 300,
    ("Gadget", 3): 295,
    ("Gadget", 4): 245,
    ("Gadget", 5): 240,

    ("Flugel", 1): 140,
    ("Flugel", 2): 175,
    ("Flugel", 3): 205,
    ("Flugel", 4): 235,
    ("Flugel", 5): 230
```

```

}

# selling price per unit
selling_price = {
    "Widget": 2490,
    "Gadget": 1990,
    "Flugel": 2970
}

# advertising cost required to create one additional unit of demand
advertising_cost = {
    "Widget": 160,
    "Gadget": 120,
    "Flugel": 180
}

# raw material 1 requirements in pounds per unit
raw_material_1 = {
    ("Plant_A", "Widget"): 194,
    ("Plant_A", "Gadget"): 230,
    ("Plant_A", "Flugel"): 178,

    ("Plant_B", "Widget"): 188,
    ("Plant_B", "Gadget"): 225,
    ("Plant_B", "Flugel"): 170
}

# raw material 2 requirements in pounds per unit - Gadgets do not require
raw material 2.
raw_material_2 = {
    ("Plant_A", "Widget"): 8.6,
    ("Plant_A", "Gadget"): 0,
    ("Plant_A", "Flugel"): 11.6,

    ("Plant_B", "Widget"): 9.2,
    ("Plant_B", "Gadget"): 0,
    ("Plant_B", "Flugel"): 10.8
}

# labor requirements in hours per unit

```

```

labor_hours = {
    ("Plant_A", "Widget"): 9.5,
    ("Plant_A", "Gadget"): 7.1,
    ("Plant_A", "Flugel"): 11.1,

    ("Plant_B", "Widget"): 9.1,
    ("Plant_B", "Gadget"): 7.8,
    ("Plant_B", "Flugel"): 10.6
}

# regular labor availability by plant and period
regular_labor_capacity = {
    ("Plant_A", period): 2500 for period in periods
}
regular_labor_capacity.update({
    ("Plant_B", period): 3800 for period in periods
})

# regular labor cost per hour
regular_labor_cost = {
    ("Plant_A", period): 11 if period <= 2 else 11 * 1.05
    for period in periods
}
regular_labor_cost.update({
    ("Plant_B", period): 11 if period <= 2 else 11 * 1.10
    for period in periods
})

# overtime labor cost per hour
overtime_labor_cost = {
    ("Plant_A", period): 16.50 if period <= 2 else 16.50 * 1.05
    for period in periods
}
overtime_labor_cost.update({
    ("Plant_B", period): 16.50 if period <= 2 else 16.50 * 1.10
    for period in periods
})

# inventory capacity by plant
inventory_capacity = {

```

```

    "Plant_A": 70,
    "Plant_B": 50
}

# inventory cost for holding one unit from one period to the next
inventory_cost = {
    ("Plant_A", "Widget"): 7.50,
    ("Plant_A", "Gadget"): 5.50,
    ("Plant_A", "Flugel"): 6.50,

    ("Plant_B", "Widget"): 7.80,
    ("Plant_B", "Gadget"): 5.70,
    ("Plant_B", "Flugel"): 7.00
}

# raw material costs by plant
raw_material_1_cost = {
    "Plant_A": 1.25,
    "Plant_B": 1.45
}

raw_material_2_cost = {
    "Plant_A": 2.65,
    "Plant_B": 2.90
}

# transportation cost per unit shipped from a plant to the distribution
center
shipping_cost = {
    ("Plant_A", "Widget"): 6.30,
    ("Plant_A", "Gadget"): 4.60,
    ("Plant_A", "Flugel"): 5.50,

    ("Plant_B", "Widget"): 6.50,
    ("Plant_B", "Gadget"): 5.00,
    ("Plant_B", "Flugel"): 5.70
}

# total limits that apply during each period
raw_material_1_capacity = 70 * 2000      # 70 tons converted to pounds

```

```
raw_material_2_capacity = 2.5 * 2000    # 2.5 tons converted to pounds
total_advertising_budget = 70000
```

D.2: Baseline Decision Variables

The baseline model uses continuous, nonnegative decision variables for production, shipment, inventory, labor, overtime, and advertising-generated demand.

```
# define production variables
production = {
    (product, plant, period): LpVariable(
        f"Production_{product}_{plant}_P{period}", 0, None
    )
    for product in products
    for plant in plants
    for period in periods
}

# define shipment variables
shipment = {
    (product, plant, period): LpVariable(
        f"Shipment_{product}_{plant}_P{period}", 0, None
    )
    for product in products
    for plant in plants
    for period in periods
}

# define inventory variables
inventory = {
    (product, plant, period): LpVariable(
        f"Inventory_{product}_{plant}_P{period}", 0, None
    )
    for product in products
    for plant in plants
    for period in periods
}

# define regular labor variables
regular_labor = {
```

```

        (plant, period): LpVariable(
            f"Regular_Labor_{plant}_P{period}",
            0,
            regular_labor_capacity[(plant, period)]
        )
    for plant in plants
    for period in periods
}

# define overtime labor variables
overtime_labor = {
    (plant, period): LpVariable(
        f"Overtime_Labor_{plant}_P{period}", 0, None
    )
    for plant in plants
    for period in periods
}

# define advertising variables
advertising = {
    (product, period): LpVariable(
        f"Advertising_Demand_{product}_P{period}", 0, None
    )
    for product in products
    for period in advertising_periods
}

```

D.3: Objective Function

```

# defines the problem
nu_model = LpProblem("NU_Industries_Baseline", LpMaximize)

# calculate total sales revenue
total_revenue = lpSum(
    selling_price[product] * shipment[(product, plant, period)]
    for product in products
    for plant in plants
    for period in periods
)

# calculate total raw material cost

```

```

total_raw_material_cost = lpSum(
    (
        raw_material_1[(plant, product)] * raw_material_1_cost[plant]
        + raw_material_2[(plant, product)] * raw_material_2_cost[plant]
    ) * production[(product, plant, period)]
    for product in products
    for plant in plants
    for period in periods
)

# calculate total regular labor cost
total_regular_labor_cost = lpSum(
    regular_labor_cost[(plant, period)] * regular_labor[(plant, period)]
    for plant in plants
    for period in periods
)

# calculate total overtime labor cost
total_overtime_labor_cost = lpSum(
    overtime_labor_cost[(plant, period)] * overtime_labor[(plant, period)]
    for plant in plants
    for period in periods
)

# calculate total inventory cost
total_inventory_cost = lpSum(
    inventory_cost[(plant, product)] * inventory[(product, plant, period)]
    for product in products
    for plant in plants
    for period in periods[:-1]
)

# calculate total transportation cost
total_shipping_cost = lpSum(
    shipping_cost[(plant, product)] * shipment[(product, plant, period)]
    for product in products
    for plant in plants
    for period in periods
)

```

```

# calculate total advertising cost
total_advertising_cost = lpSum(
    advertising_cost[product] * advertising[(product, period)]
    for product in products
    for period in advertising_periods
)

# define objective function
nu_model += (
    total_revenue
    - total_raw_material_cost
    - total_regular_labor_cost
    - total_overtime_labor_cost
    - total_inventory_cost
    - total_shipping_cost
    - total_advertising_cost
)

```

D.4: Model Constraints

The following constraints enforce inventory balance, demand satisfaction, labor coverage, storage capacity, raw material availability, the advertising budget, and zero ending inventory.

```

# inventory balance constraints
# beginning inventory + production = shipment + ending inventory
for product in products:
    for plant in plants:
        for period in periods:
            if period == 1:
                nu_model += (
                    production[(product, plant, period)]
                    == shipment[(product, plant, period)]
                    + inventory[(product, plant, period)]
                ), f"Inventory_Balance_{product}_{plant}_P{period}"
            else:
                nu_model += (
                    inventory[(product, plant, period - 1)]
                    + production[(product, plant, period)]
                    == shipment[(product, plant, period)]
                    + inventory[(product, plant, period)]
                ), f"Inventory_Balance_{product}_{plant}_P{period}"

```

```

# demand constraints
# Period 1 only includes the required contract demand and then Periods 2
# through 5 include contract demand plus demand created by prior
# advertising.
for product in products:
    for period in periods:
        if period == 1:
            required_demand = base_demand[(product, period)]
        else:
            required_demand = (
                base_demand[(product, period)]
                + advertising[(product, period - 1)]
            )

        nu_model += (
            lpSum(
                shipment[(product, plant, period)]
                for plant in plants
            )
            == required_demand
        ), f"Demand_{product}_P{period}"

# labor balance constraints
# Production labor must be covered by regular labor and overtime labor
for plant in plants:
    for period in periods:
        nu_model += (
            lpSum(
                labor_hours[(plant, product)]
                * production[(product, plant, period)]
                for product in products
            )
            == regular_labor[(plant, period)]
            + overtime_labor[(plant, period)]
        ), f"Labor_Balance_{plant}_P{period}"

# inventory storage capacity constraints
for plant in plants:
    for period in periods[:-1]:
        nu_model += (

```

```

        lpSum(
            inventory[(product, plant, period)]
            for product in products
        )
        <= inventory_capacity[plant]
    ), f"Inventory_Capacity_{plant}_P{period}"

# raw material 1 constraints
# The vendor can provide a total of 70 tons across both plants during each
period
for period in periods:
    nu_model += (
        lpSum(
            raw_material_1[(plant, product)]
            * production[(product, plant, period)]
            for product in products
            for plant in plants
        )
        <= raw_material_1_capacity
    ), f"Raw_Material_1_P{period}"

# raw material 2 constraints
# The vendor can provide a total of 2.5 tons across both plants during
each period
for period in periods:
    nu_model += (
        lpSum(
            raw_material_2[(plant, product)]
            * production[(product, plant, period)]
            for product in products
            for plant in plants
        )
        <= raw_material_2_capacity
    ), f"Raw_Material_2_P{period}"

# total advertising budget constraint
nu_model += (
    lpSum(
        advertising_cost[product] * advertising[(product, period)]
        for product in products
    )

```

```

        for period in advertising_periods
    )
    <= total_advertising_budget
), "Advertising_Budget"

# There is no inventory at the end of the planning horizon
for product in products:
    for plant in plants:
        nu_model += (
            inventory[(product, plant, 5)] == 0
        ), f"Ending_Inventory_{product}_{plant}"

```

D.5: Baseline Model Solution

```

# solve the problem
nu_model.writeLP("NU_Industries_Baseline.lp")
nu_model.solve(GLPK(msg=False))
print("Status:", LpStatus[nu_model.status])

print("Maximum Profit = $", format(round(value(nu_model.objective), 1),
","))

```

The GLPK solver returned an optimal status and baseline maximum profit of approximately \$7,066,036.8.

D.6: Model Verification and Validation Code

D.6.1: Advertising Budget Validation

```

# validate advertising budget
advertising_spending = value(total_advertising_cost)

advertising_validation = pd.DataFrame({
    "Budget Used": [advertising_spending],
    "Budget Limit": [total_advertising_budget],
    "Remaining Budget": [total_advertising_budget - advertising_spending],
    "Status": [
        "PASS" if advertising_spending <= total_advertising_budget else
"FAIL"
    ]
})

advertising_validation.round(1)

```

D.6.2: Raw Material Capacity Validation

```
# validate raw material capacity
#tolerance = 1e-6
# used 0.5 for small solver rounding differences
tolerance = 0.5

raw_material_validation = raw_material_results.copy()

raw_material_validation["Raw Material 1 Status"] =
raw_material_validation.apply(
    lambda row: "PASS"
    if row["Raw Material 1 Used"] <= row["Raw Material 1 Capacity"] +
tolerance
    else "FAIL",
    axis=1
)

raw_material_validation["Raw Material 2 Status"] =
raw_material_validation.apply(
    lambda row: "PASS"
    if row["Raw Material 2 Used"] <= row["Raw Material 2 Capacity"] +
tolerance
    else "FAIL",
    axis=1
)

raw_material_validation.round(1)
```

D.6.3: Regular Labor Capacity Validation

```
# validate labor capacity
labor_validation = labor_results.copy()

labor_validation["Regular Labor Capacity"] = labor_validation.apply(
    lambda row: regular_labor_capacity[(row["Plant"], row["Period"])],
    axis=1
)

labor_validation["Status"] = labor_validation.apply(
    lambda row: "PASS"
    if row["Regular Labor"] <= row["Regular Labor Capacity"] + tolerance
```

```

        else "FAIL",
        axis=1
    )

labor_validation.round(1)

```

D.6.4: Ending Inventory Validation

```

# validate ending inventory
ending_inventory_results = []

for product in products:
    for plant in plants:
        ending_inventory_results.append({
            "Product": product,
            "Plant": plant,
            "Ending Inventory": inventory[(product, plant, 5)].varValue,
            "Status": (
                "PASS"
                if abs(inventory[(product, plant, 5)].varValue) <=
tolerance
                else "FAIL"
            )
        })

ending_inventory_validation = pd.DataFrame(ending_inventory_results)

ending_inventory_validation.round(1)

```

D.6.5: Inventory Capacity Validation

```

# validate inventory capacity
inventory_capacity_results = []

for plant in plants:
    for period in periods[:4]:
        total_inventory = sum(
            inventory[(product, plant, period)].varValue
            for product in products
        )

        inventory_capacity_results.append({

```

```

        "Plant": plant,
        "Period": period,
        "Inventory Used": total_inventory,
        "Inventory Capacity": inventory_capacity[plant],
        "Status": (
            "PASS"
            if total_inventory <= inventory_capacity[plant] +
tolerance
            else "FAIL"
        )
    ))

inventory_capacity_validation = pd.DataFrame(inventory_capacity_results)

inventory_capacity_validation.round(1)

```

D.6.6: Demand Satisfaction Validation

```

# validate demand satisfaction
demand_validation_results = []

for product in products:
    for period in periods:
        total_shipments = sum(
            shipment[(product, plant, period)].varValue
            for plant in plants
        )

        base_required_demand = base_demand[(product, period)]

        if period == 1:
            advertising_demand = 0
        else:
            advertising_demand = advertising[(product, period -
1)].varValue

        required_demand = base_required_demand + advertising_demand

        demand_validation_results.append({
            "Product": product,
            "Period": period,

```

```

        "Base Demand": base_required_demand,
        "Advertising Demand": advertising_demand,
        "Required Demand": required_demand,
        "Shipments": total_shipments,
        "Status": (
            "PASS"
            if abs(total_shipments - required_demand) <= tolerance
            else "FAIL"
        )
    })

demand_validation = pd.DataFrame(demand_validation_results)

demand_validation.round(1)

```

D.6.7: Inventory Balance Validation

```

# validate inventory balance
inventory_balance_results = []

for product in products:
    for plant in plants:
        for period in periods:

            if period == 1:
                beginning_inventory = 0
            else:
                beginning_inventory = inventory[(product, plant, period -
1)].varValue

            production_value = production[(product, plant,
period)].varValue
            shipment_value = shipment[(product, plant, period)].varValue
            ending_inventory = inventory[(product, plant,
period)].varValue

            difference = (
                beginning_inventory
                + production_value
                - shipment_value
                - ending_inventory

```

```

    )

    inventory_balance_results.append({
        "Product": product,
        "Plant": plant,
        "Period": period,
        "Beginning Inventory": beginning_inventory,
        "Production": production_value,
        "Shipment": shipment_value,
        "Ending Inventory": ending_inventory,
        "Difference": difference,
        "Status": (
            "PASS"
            if abs(difference) <= tolerance
            else "FAIL"
        )
    })

inventory_balance_df = pd.DataFrame(inventory_balance_results)

inventory_balance_df.round(1)

```

D.7: Scenario Analysis

D.7.1: Increased Advertising Budget

```

# save baseline results
baseline_profit = value(nu_model.objective)
baseline_advertising_spending = value(total_advertising_cost)

# increase advertising budget to $100,000
nu_model.constraints["Advertising_Budget"].changeRHS(100000)

# solve the scenario
nu_model.solve(GLPK(msg=False))

scenario_1_profit = value(nu_model.objective)
scenario_1_advertising_spending = value(total_advertising_cost)

scenario_1_results = pd.DataFrame({
    "Measure": [
        "Advertising Budget",

```

```

        "Advertising Spending",
        "Maximum Profit"
    ],
    "Baseline": [
        total_advertising_budget,
        baseline_advertising_spending,
        baseline_profit
    ],
    "Scenario 1": [
        100000,
        scenario_1_advertising_spending,
        scenario_1_profit
    ]
})

scenario_1_results["Change"] = (
    scenario_1_results["Scenario 1"]
    - scenario_1_results["Baseline"]
)

scenario_1_results.round(1)

# restore baseline advertising budget
nu_model.constraints["Advertising_Budget"].changeRHS(70000)

# solve the baseline model again
nu_model.solve(GLPK(msg=False))

print("Status:", LpStatus[nu_model.status])

```

D.7.2: Increased Raw Material Availability

```

# save baseline profit
baseline_profit = value(nu_model.objective)

# Scenario 2A: increase Raw Material 1 by 25 percent
for period in periods:
    nu_model.constraints[f"Raw_Material_1_P{period}"].changeRHS(175000)

nu_model.solve(GLPK(msg=False))
rm1_profit = value(nu_model.objective)

```

```

# restore Raw Material 1 to baseline
for period in periods:
    nu_model.constraints[f"Raw_Material_1_P{period}"].changeRHS(140000)

# Scenario 2B: increase Raw Material 2 by 25 percent
for period in periods:
    nu_model.constraints[f"Raw_Material_2_P{period}"].changeRHS(6250)

nu_model.solve(GLPK(msg=False))

rm2_profit = value(nu_model.objective)

# restore Raw Material 2 to baseline
for period in periods:
    nu_model.constraints[f"Raw_Material_2_P{period}"].changeRHS(5000)

# return model to baseline
nu_model.solve(GLPK(msg=False))

scenario_2_results = pd.DataFrame({
    "Scenario": [
        "Baseline",
        "RM1 +25%",
        "RM2 +25%"
    ],
    "Maximum Profit": [
        baseline_profit,
        rm1_profit,
        rm2_profit
    ],
    "Profit Increase": [
        0,
        rm1_profit - baseline_profit,
        rm2_profit - baseline_profit
    ]
})

scenario_2_results.round(1)

```

D.7.3: Increased Regular Labor Capacity

```
# save baseline profit
baseline_profit = value(nu_model.objective)

# Scenario 3A: increase Plant A regular labor capacity by 25 percent
for period in periods:
    regular_labor[("Plant_A", period)].upBound = 3125

nu_model.solve(GLPK(msg=False))

plant_a_labor_profit = value(nu_model.objective)

# restore Plant A regular labor capacity
for period in periods:
    regular_labor[("Plant_A", period)].upBound = 2500

# Scenario 3B: increase Plant B regular labor capacity by 25 percent
for period in periods:
    regular_labor[("Plant_B", period)].upBound = 4750

nu_model.solve(GLPK(msg=False))

plant_b_labor_profit = value(nu_model.objective)

# restore Plant B regular labor capacity
for period in periods:
    regular_labor[("Plant_B", period)].upBound = 3800

# return model to baseline
nu_model.solve(GLPK(msg=False))

scenario_3_results = pd.DataFrame({
    "Scenario": [
        "Baseline",
        "Plant A Labor +25%",
        "Plant B Labor +25%"
    ],
    "Maximum Profit": [
        baseline_profit,
        plant_a_labor_profit,
```

```

        plant_b_labor_profit
    ],
    "Profit Increase": [
        0,
        plant_a_labor_profit - baseline_profit,
        plant_b_labor_profit - baseline_profit
    ]
})

scenario_3_results.round(1)

```

D.7.4: Combined Bottleneck Relief

```

# Scenario 4: increase RM1 by 25% and Plant B regular labor by 25%

# save baseline profit
baseline_profit = value(nu_model.objective)

# increase Raw Material 1 availability by 25%
for period in periods:
    nu_model.constraints[f"Raw_Material_1_P{period}"].changeRHS(175000)

# increase Plant B regular labor capacity by 25%
for period in periods:
    regular_labor[("Plant_B", period)].upBound = 4750

# solve scenario
nu_model.solve(GLPK(msg=False))

combined_profit = value(nu_model.objective)

# compare results
scenario_4_results = pd.DataFrame({
    "Scenario": [
        "Baseline",
        "RM1 +25% and Plant B Labor +25%"
    ],
    "Maximum Profit": [
        baseline_profit,
        combined_profit
    ],

```

```

        "Profit Increase": [
            0,
            combined_profit - baseline_profit
        ]
    })

scenario_4_results.round(1)

# restore baseline RM1 availability
for period in periods:
    nu_model.constraints[f"Raw_Material_1_P{period}"].changeRHS(140000)

# restore baseline Plant B regular labor capacity
for period in periods:
    regular_labor[("Plant_B", period)].upBound = 3800

# return model to baseline
nu_model.solve(GLPK(msg=False))
print("Model reset status:", IpStatus[nu_model.status])

```

D.8: Raw Material 1 Sensitivity Analysis

```

# test several increases in Raw Material 1 availability
rm1_sensitivity_results = []

rm1_levels = {
    "Baseline": 140000,
    "+5%": 147000,
    "+10%": 154000,
    "+15%": 161000,
    "+20%": 168000,
    "+25%": 175000,
    "+50%": 210000
}

for label, capacity in rm1_levels.items():

    for period in periods:

nu_model.constraints[f"Raw_Material_1_P{period}"].changeRHS(capacity)

```

```

nu_model.solve(GLPK(msg=False))

scenario_profit = value(nu_model.objective)

rm1_sensitivity_results.append({
    "RM1 Level": label,
    "RM1 Capacity": capacity,
    "Maximum Profit": scenario_profit,
    "Profit Increase": scenario_profit - baseline_profit
})

# restore baseline RM1 availability
for period in periods:
    nu_model.constraints[f"Raw_Material_1_P{period}"].changeRHS(140000)

nu_model.solve(GLPK(msg=False))

rm1_sensitivity_df = pd.DataFrame(rm1_sensitivity_results)

rm1_sensitivity_df.round(1)

# increase Raw Material 1 availability by 5 percent
for period in periods:
    nu_model.constraints[f"Raw_Material_1_P{period}"].changeRHS(147000)

nu_model.solve(GLPK(msg=False))

# compare raw material usage with available capacity
rm1_followup_results = []

for period in periods:

    rm1_used = sum(
        raw_material_1[(plant, product)]
        * production[(product, plant, period)].varValue
        for plant in plants
        for product in products
    )

    rm2_used = sum(

```

```

        raw_material_2[(plant, product)]
        * production[(product, plant, period)].varValue
    for plant in plants
    for product in products
)

rm1_followup_results.append({
    "Period": period,
    "RM1 Used": rm1_used,
    "RM1 Capacity": 147000,
    "RM1 Slack": 147000 - rm1_used,
    "RM2 Used": rm2_used,
    "RM2 Capacity": 5000,
    "RM2 Slack": 5000 - rm2_used
})

rm1_followup_df = pd.DataFrame(rm1_followup_results)

rm1_followup_df.round(1)

```

D.9: Integer Programming Model

```

# calculate total sales revenue
integer_total_revenue = lpSum(
    selling_price[product] * integer_shipment[(product, plant, period)]
    for product in products
    for plant in plants
    for period in periods
)

integer_total_raw_material_cost = lpSum(
    (
        raw_material_1[(plant, product)] * raw_material_1_cost[plant]
        + raw_material_2[(plant, product)] * raw_material_2_cost[plant]
    ) * integer_production[(product, plant, period)]
    for product in products
    for plant in plants
    for period in periods
)

integer_total_regular_labor_cost = lpSum(

```

```

        regular_labor_cost[(plant, period)]
        * integer_regular_labor[(plant, period)]
        for plant in plants
        for period in periods
    )

integer_total_overtime_labor_cost = lpSum(
    overtime_labor_cost[(plant, period)]
    * integer_overtime_labor[(plant, period)]
    for plant in plants
    for period in periods
)

integer_total_inventory_cost = lpSum(
    inventory_cost[(plant, product)]
    * integer_inventory[(product, plant, period)]
    for product in products
    for plant in plants
    for period in periods[:-1]
)

integer_total_shipping_cost = lpSum(
    shipping_cost[(plant, product)]
    * integer_shipment[(product, plant, period)]
    for product in products
    for plant in plants
    for period in periods
)

integer_total_advertising_cost = lpSum(
    advertising_cost[product]
    * integer_advertising[(product, period)]
    for product in products
    for period in advertising_periods
)

integer_model += (
    integer_total_revenue
    - integer_total_raw_material_cost
    - integer_total_regular_labor_cost

```

```

- integer_total_overtime_labor_cost
- integer_total_inventory_cost
- integer_total_shipping_cost
- integer_total_advertising_cost
)

# inventory balance constraints
for product in products:
    for plant in plants:
        for period in periods:
            if period == 1:
                integer_model += (
                    integer_production[(product, plant, period)]
                    == integer_shipment[(product, plant, period)]
                    + integer_inventory[(product, plant, period)]
                )
            else:
                integer_model += (
                    integer_inventory[(product, plant, period - 1)]
                    + integer_production[(product, plant, period)]
                    == integer_shipment[(product, plant, period)]
                    + integer_inventory[(product, plant, period)]
                )

# demand constraints
for product in products:
    for period in periods:
        if period == 1:
            required_demand = base_demand[(product, period)]
        else:
            required_demand = (
                base_demand[(product, period)]
                + integer_advertising[(product, period - 1)]
            )

        integer_model += (
            lpSum(
                integer_shipment[(product, plant, period)]
                for plant in plants
            )
            == required_demand

```

```

    )

# labor balance constraints
for plant in plants:
    for period in periods:
        integer_model += (
            lpSum(
                labor_hours[(plant, product)]
                * integer_production[(product, plant, period)]
                for product in products
            )
            == integer_regular_labor[(plant, period)]
            + integer_overtime_labor[(plant, period)]
        )

# inventory capacity constraints
for plant in plants:
    for period in periods[:-1]:
        integer_model += (
            lpSum(
                integer_inventory[(product, plant, period)]
                for product in products
            )
            <= inventory_capacity[plant]
        )

# raw material 1 constraints
for period in periods:
    integer_model += (
        lpSum(
            raw_material_1[(plant, product)]
            * integer_production[(product, plant, period)]
            for product in products
            for plant in plants
        )
        <= raw_material_1_capacity
    )

# raw material 2 constraints
for period in periods:

```

```

integer_model += (
    lpSum(
        raw_material_2[(plant, product)]
        * integer_production[(product, plant, period)]
        for product in products
        for plant in plants
    )
    <= raw_material_2_capacity
)

# advertising budget constraint
integer_model += (
    lpSum(
        advertising_cost[product]
        * integer_advertising[(product, period)]
        for product in products
        for period in advertising_periods
    )
    <= total_advertising_budget
)

# ending inventory must be zero
for product in products:
    for plant in plants:
        integer_model += (
            integer_inventory[(product, plant, 5)] == 0
        )

# solve integer model with a 60-second time limit
integer_model.solve(
    GLPK(
        msg=True,
        options=["--tmlim", "60"]
    )
)

# compare integer solution with baseline
integer_profit = value(integer_model.objective)

print("Integer model status:", LpStatus[integer_model.status])

```

```
print("Integer model profit:", round(integer_profit, 1))
print("Baseline LP profit:", round(baseline_profit, 1))
print("Profit difference:", round(integer_profit - baseline_profit, 1))

# double-check to make sure that production quantities are whole units
integer_production_check = all(
    abs(value.varValue - round(value.varValue)) < 1e-6
    for value in integer_production.values())

print("All production quantities are integers:", integer_production_check)
```